

BUILDING DISTRIBUTED AGENT-BASED MODELS WITH REPAST4PY

NICK COLLIER

Senior Software Engineer
Argonne National Laboratory

JONATHAN OZIK

Principal Computational Scientist
Argonne National Laboratory



U.S. DEPARTMENT OF
ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

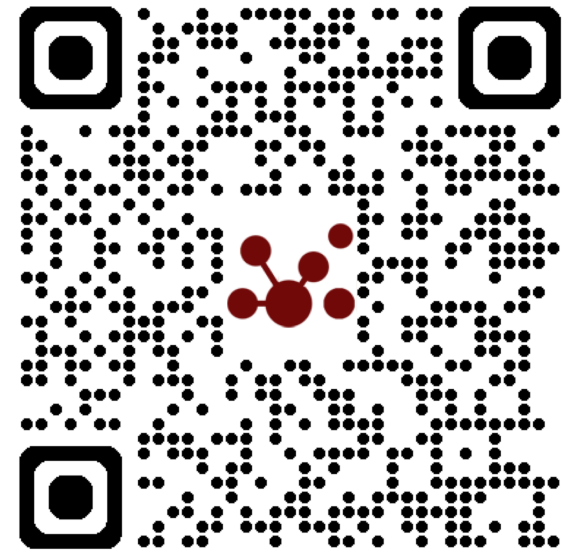
25 August 2025
SSC 2025 Delft

OUTLINE

- Introduction and Motivation
- Repast4Py Overview
- Repast4Py Distributed ABM Components
- Building a Model: Hands-on Coding Exercise
- Performance Techniques

Repast4Py

Distributed ABM for everyone



JUPYTER LAB ON BINDER

<https://tinyurl.com/dn37dzuk>

Full URL:

https://mybinder.org/v2/gh/Repast/repast4py-tutorial/ssc_2025

INTRODUCTION AND MOTIVATION



**U.S. DEPARTMENT OF
ENERGY**

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

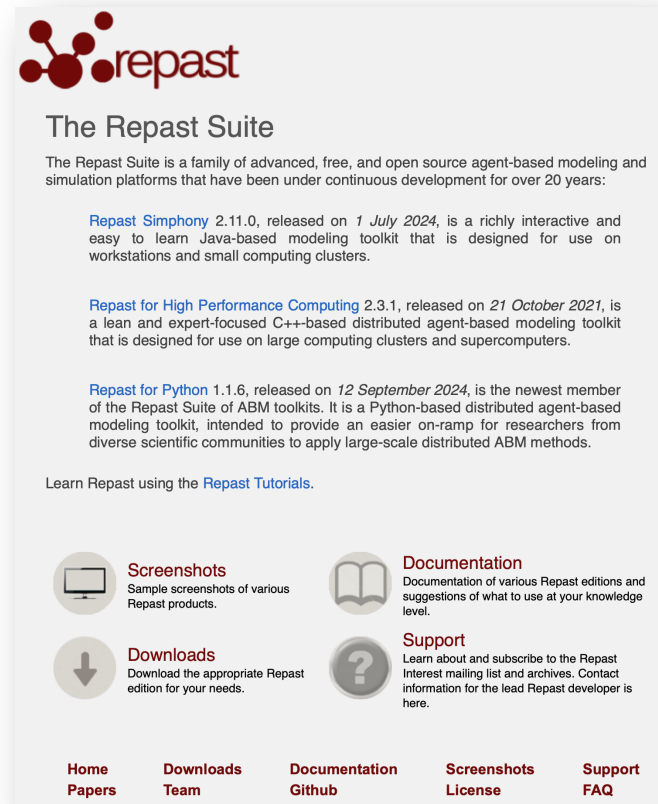
Argonne 
NATIONAL LABORATORY

REPAST AGENT-BASED MODELING SUITE



<https://repast.github.io>

- Family of advanced, free, and open source agent-based modeling and simulation platforms that have collectively been under continuous development for over 20 years
- Over 200,000 downloads globally
- Developed and maintained at Argonne
- Three flavors of agent-based modeling toolkits
 - Repast Symphony
 - Repast for High Performance Computing (Repast HPC)
 - Repast4Py



The Repast Suite

The Repast Suite is a family of advanced, free, and open source agent-based modeling and simulation platforms that have been under continuous development for over 20 years:

[Repast Symphony](#) 2.11.0, released on 1 July 2024, is a richly interactive and easy to learn Java-based modeling toolkit that is designed for use on workstations and small computing clusters.

[Repast for High Performance Computing](#) 2.3.1, released on 21 October 2021, is a lean and expert-focused C++-based distributed agent-based modeling toolkit that is designed for use on large computing clusters and supercomputers.

[Repast for Python](#) 1.1.6, released on 12 September 2024, is the newest member of the Repast Suite of ABM toolkits. It is a Python-based distributed agent-based modeling toolkit, intended to provide an easier on-ramp for researchers from diverse scientific communities to apply large-scale distributed ABM methods.

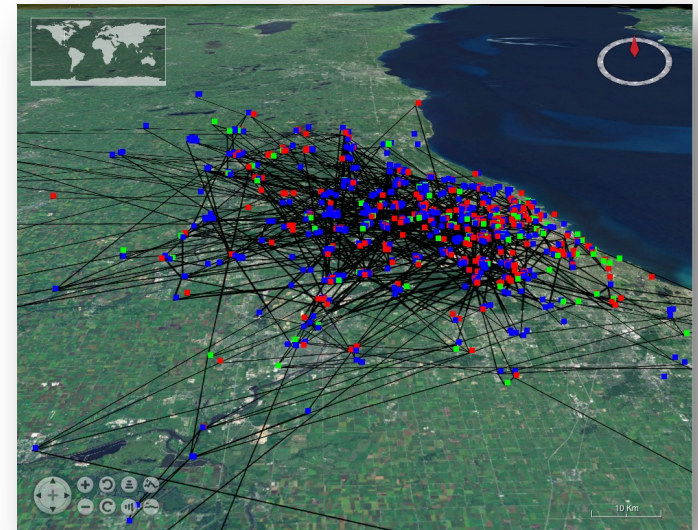
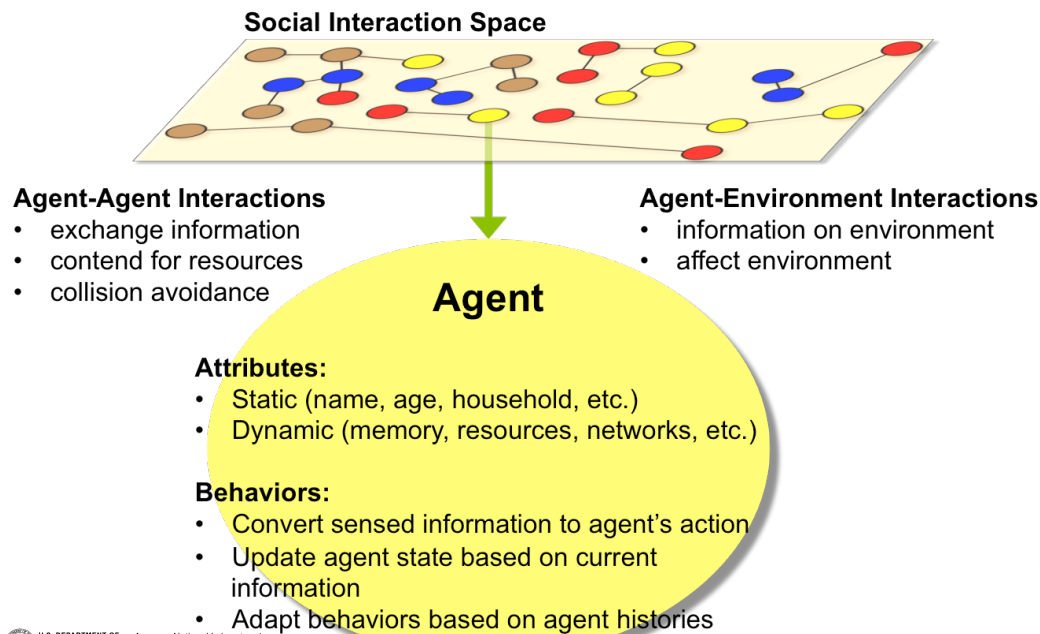
Learn Repast using the [Repast Tutorials](#).

 Screenshots Sample screenshots of various Repast products.	 Documentation Documentation of various Repast editions and suggestions of what to use at your knowledge level.
 Downloads Download the appropriate Repast edition for your needs.	 Support Learn about and subscribe to the Repast Interest mailing list and archives. Contact information for the lead Repast developer is here .

Home Papers	Downloads Team	Documentation Github	Screenshots License	Support FAQ
-----------------------------	--------------------------------	--------------------------------------	-------------------------------------	-----------------------------

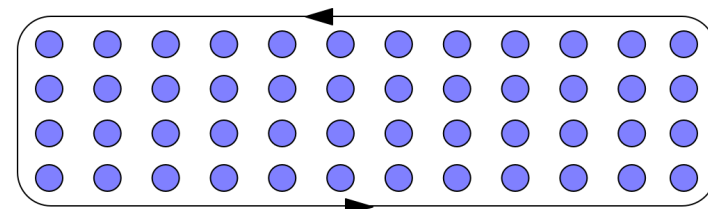
AGENT-BASED MODELS (ABMS)

- **Disaggregated description of complex systems:**
 - Method of computing the potential system-level consequences of the behaviors of sets of individuals
 - Effects of interventions can be run with different assumptions

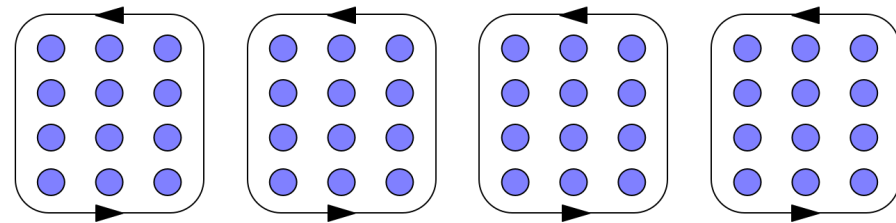


MOTIVATION FOR DISTRIBUTED PARALLEL ABMS

- Current desktop / laptop CPUs typically contain multiple processing cores.
- HPC clusters and supercomputers can contain thousands or more cores and increasingly, also GPUs
- Distributed ABM
 - partitions the agents over multiple processes, resulting in a *faster runtime*.
 - allows for greater numbers of agents and more complex behavior.



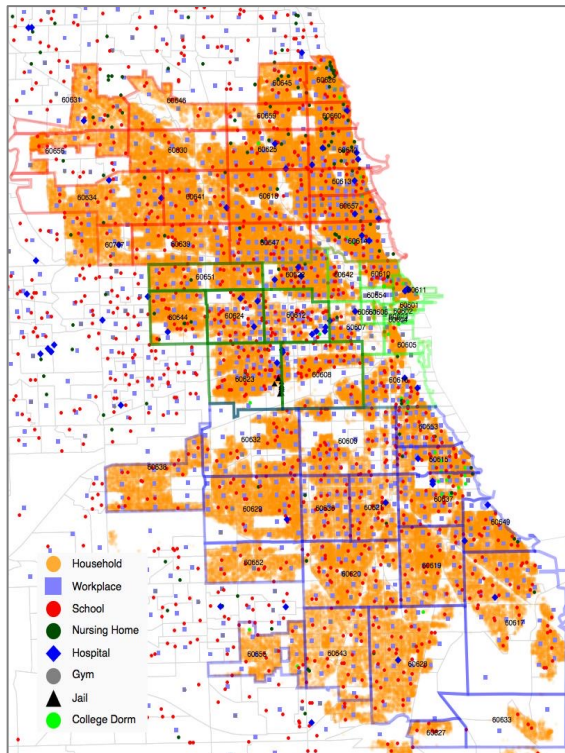
Agents Executing Sequentially on a Single Process



Agents Executing Sequentially on Multiple Concurrent Processes

ChiSIM: THE CHICAGO SOCIAL INTERACTION MODEL

SIMULATING CHICAGO AND EVERYONE IN IT



- 2.7(-8.5) million individual people agents
- Moving to/from 1.2(-5.2) million geo-located places on an hourly basis according to individual activity schedules
- Agents interact with other co-located agents at those places

Macal, C. M., N. T. Collier, J. Ozik, E. R. Tatara, and J. T. Murphy. 2018. "CHISIM: AN AGENT-BASED SIMULATION MODEL OF SOCIAL INTERACTIONS IN A LARGE URBAN AREA." In *2018 Winter Simulation Conference (WSC)*, 810–20. <https://doi.org/10.1109/WSC.2018.8632409>.



67 Days to Lockdown

From Chicago's first known COVID-19 case to the governor's historic shelter-in-place order, here's how those critical, frenzied early weeks of the pandemic unfolded, in the words of those at the center of the crisis.

BY MIKE THOMAS
ILLUSTRATION BY KEITH NEGLEY
MARCH 2, 2021, 8:00 AM

<https://www.chicagomag.com/chicago-magazine/march-2021/67-days-to-lockdown/>



ARWADY

Commissioner of the Chicago Department of
Public Health

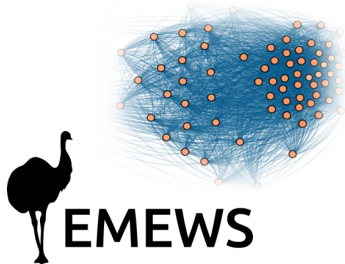
Argonne National Lab built what's called an agent-based model specifically for Chicago that predated COVID and mimics the movement of its 2.7 million residents. All the gyms, schools, and businesses are built in, and it [takes into account] how people interact throughout the day. So pretty early on, we connected with them to start thinking about using that Chicago model for COVID.

MONDAY, MARCH 9

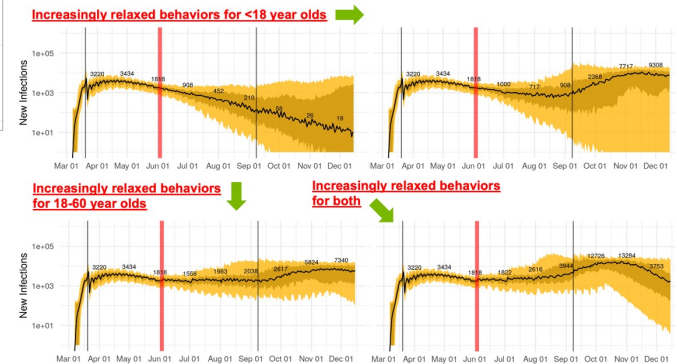
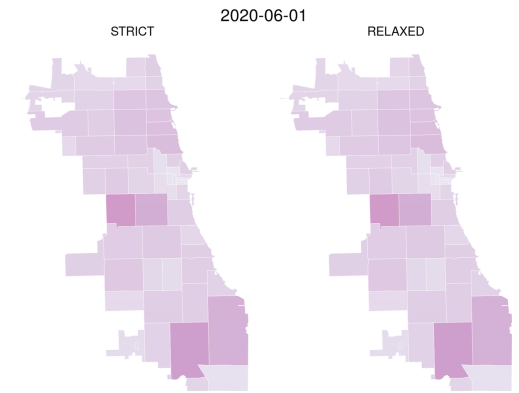
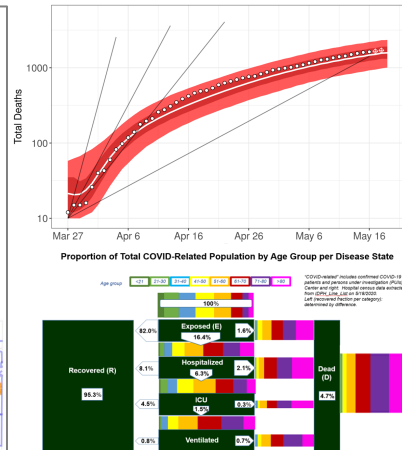
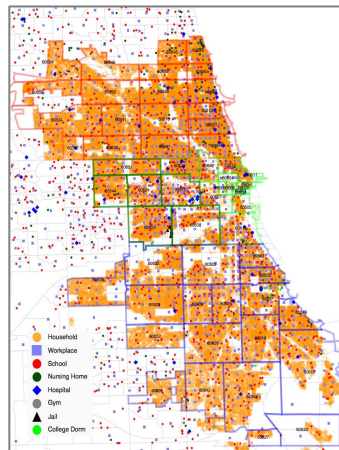
The State's Disaster Proclamation

One day after a Cook County man was announced as Illinois's seventh case and its first known instance of "community spread" — the spread of a disease to people who have had no known contact with other infected people — Pritzker issued a disaster proclamation that marshaled state resources to combat the virus. Four additional cases were revealed this day, all of them in Cook County. The next day saw eight more cases, including the first two outside Cook, bringing the state total to 19. The numbers were rising fast now, just as experts had predicted.

RAPIDLY EVOLVING POLICY AND EPIDEMIOLOGY QUESTIONS



CityCOVID model for Chicago, 2.7 M people (agents) move hourly between 1.2 M locations



What mitigation strategies should be considered?

How will COVID-19 affect populations?

March

How should we ease mitigations?

How does mobility affect transmission?

April

What are place/occupation based risks?

How do behaviors affect transmission?

May

How can we reopen schools/universities?

How do different age groups behave differently?

June

Policy

Epidemiology

Argonne NATIONAL LABORATORY

Special Issue Papers

A population data-driven workflow for COVID-19 modeling and learning

2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)



Developing Distributed High-performance Computing Capabilities of an Open Science Platform for Robust Epidemic Analysis

Nicholson Collier
Decision and Infrastructure Sciences
Argonne National Laboratory
Lemont, IL, U.S.A.
<https://orcid.org/0000-0002-2376-4156>

Justin M. Wozniak
Data Science and Learning
Argonne National Laboratory
Lemont, IL, U.S.A.

Abby Stevens
Decision and Infrastructure Sciences
Argonne National Laboratory
Lemont, IL, U.S.A.

Yadu Babuji
Computer Science
University of Chicago
Chicago, IL, U.S.A.
<https://orcid.org/0000-0002-9162-6003>

Alexandra Würth
Acumes project-team
Inria centre at Université Côte d'Azur
Sophia Antipolis, France
<https://orcid.org/0000-0002-6099-0531>



SC20

PROGRAM EXHIBITS SCINET ATTEND SUBMIT REGISTER

A Population Data-Driven Workflow for COVID-19 Modeling and Learning

Session: Gordon Bell COVID-19 Prize Finalist Session 1

Authors: Jonathan Ozik, Justin M. Wozniak, Nicholson Collier, Charles M. Macal, Mickaël Binois

Event Type: ACM Gordon Bell COVID Finalist, Awards Presentation

Registration Categories:

Time: Thursday, 19 November 2020, 11:30am - 12pm EST

AERO: An autonomous platform for continuous research

Valérie Hayot-Sasson^{1,2}, Abby Stevens², Nicholson Collier², Sudershan Sridhar³, Kyle Conroy¹, J. Gregory Pauloski^{1,2}, Yadu Babuji¹, Maxime Gonthier^{1,2}, Nathaniel Hudson^{1,2}, Dante D. Sanchez-Gallegos⁴, Ian Foster^{1,2}, Jonathan Ozik^{1,2}, Kyle Chard^{1,2}

1 University of Chicago, Chicago, Illinois, USA.
2 Argonne National Laboratory, Lemont, Illinois, USA.
3 Globus, Chicago, Illinois, USA.
4 University Carlos III of Madrid, Leganes, Madrid, Spain.

Readers' and Editors' Choice Awards

REPAST4PY OVERVIEW



**U.S. DEPARTMENT OF
ENERGY**

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne 
NATIONAL LABORATORY

REPAST4PY

- Repast4Py is an agent-based modeling framework written in Python that provides the ability to build large, **distributed agent-based models** (ABMs) that span multiple processing cores.
- Repast4Py is released under the **BSD-3 open source license**, and leverages the MPI for Python, Numba, NumPy, CuPy, and PyTorch packages, and the Python C API to create a scalable modeling system that can exploit the **largest HPC resources** and emerging computing architectures.
- **Homepage:** <https://repast.github.io/repast4py.site/index.html>
- **User Guide:** https://repast.github.io/repast4py.site/guide/user_guide.html
- **API Docs:** <https://repast.github.io/repast4py.site/apidoc/index.html>
- **Github Repo:** <https://github.com/Repast/repast4py>

Table of Contents

1. Getting Started
 - 1.1. Requirements
 - 1.2. Installation
 - 1.3. Documentation
 - 1.4. Contact and Support
2. Why Repast4Py?
 - 2.1. Distributed computing a natural fit for agent-based modeling
 - 2.2. Repast4Py and the broader Repast family
3. Repast Simulation Overview
 - 3.1. Contexts and Projections
 - 3.2. Scheduling Events
 - 3.3. Distributed Simulation
4. Cross-Process Code Requirements
 - 4.1. Agent ID
 - 4.2. Saving and Restoring Agents
 - 4.3. Synchronization
5. Tutorial 1 - A Simple Random Walk Model
 - 5.1. The Walker Agent
 - 5.2. The Model Class
 - 5.3. Restoring Walkers
 - 5.4. Running the Simulation
6. Tutorial 2 - The Rumor Network Model
 - 6.1. Overview
 - 6.2. The Network
 - 6.3. The Rumor Model Implementation
7. Tutorial 3 - The Zombies Model
 - 7.1. The Agent Classes
 - 7.2. The Model class
 - 7.3. The Grid Neighborhood Finder
 - 7.4. Running the Simulation

Repast for Python (Repast4Py) User Guide

Version 2.1 September 2024

1. Getting Started

Repast for Python (Repast4Py) is the newest member of the [Repast Suite](#) of free and open source agent-based modeling and simulation software. It builds on [Repast HPC](#), and provides the ability to build large, distributed agent-based models (ABMs) that span multiple processing cores. Distributed ABMs enable the development of complex systems models that capture the scale and relevant details of many problems of societal importance.^{[1][2]} Where Repast HPC is implemented in C++ and is more HPC expert focused, Repast4Py is a Python package and is designed to provide an easier on-ramp for researchers from diverse scientific communities to apply large-scale distributed ABM methods. Repast4Py is released under the BSD-3 open source license, and leverages [Numba](#), [NumPy](#), and [PyTorch](#) packages, and the Python C API to create a scalable modeling system that can exploit the largest HPC resources and emerging computing architectures. See our paper on Repast4Py for additional information about the design and implementation.^[3]

1.1. Requirements

Repast4Py can run on Linux, macOS and Windows provided there is a working MPI implementation installed and mpi4py is supported. Repast4Py is developed and tested on Linux. We recommend that Windows users use the Windows Subsystem for Linux (WSL). Installation instructions for WSL can be found [here](#).

Under Linux, MPI can be installed using your OS's package manager. For example, under Ubuntu 22.04 (and thus WSL), the mpich MPI implementation can be installed with:

```
$ sudo apt install mpich
```

Installation instructions for MPI on macOS can be found [here](#).

A typical campus cluster, or HPC resource will have MPI and mpi4py installed. Check the resource's documentation on available software for more details.

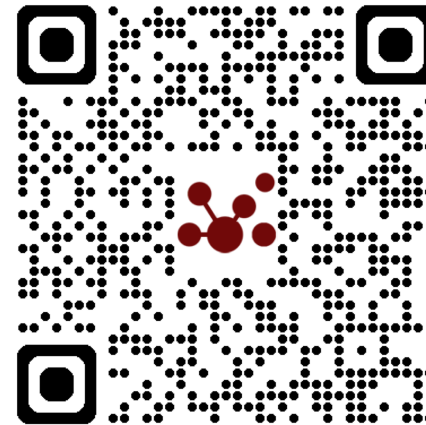
1.2. Installation

REPAST4PY

- Repast4Py is an agent-based modeling framework written in Python that provides the ability to build large, **distributed agent-based models** (ABMs) that span multiple processing cores.
- Repast4Py is released under the **BSD-3 open source license**, and leverages the MPI for Python, Numba, NumPy, CuPy, and PyTorch packages, and the Python C API to create a scalable modeling system that can exploit the **largest HPC resources** and emerging computing architectures.
- **Homepage:** <https://repast.github.io/repast4py.site/index.html>
- **User Guide:** https://repast.github.io/repast4py.site/guide/user_guide.html
- **API Docs:** <https://repast.github.io/repast4py.site/apidoc/index.html>
- **Github Repo:** <https://github.com/Repast/repast4py>

REPAST4PY: DISTRIBUTED ABM FOR EVERYONE

- **User Guide:**
 - Complete tutorials to describe how to build different types of distributed ABMs



Collier, N. T., J. Ozik, and E. R. Tatara. 2020. “Experiences in Developing a Distributed Agent-Based Modeling Toolkit with Python.” In *2020 IEEE/ACM 9th Workshop on Python for High-Performance and Scientific Computing (PyHPC)*, 1–12.
<https://doi.org/10.1109/PyHPC51966.2020.00006>.

Collier, Nicholson, and Jonathan Ozik. 2022. “Distributed Agent-Based Simulation with Repast4Py.” In *2022 Winter Simulation Conference (WSC)*, 192–206. Singapore: IEEE.
<https://doi.org/10.1109/WSC57314.2022.10015389>.

REPAST4PY DISTRIBUTED ABM COMPONENTS



**U.S. DEPARTMENT OF
ENERGY**

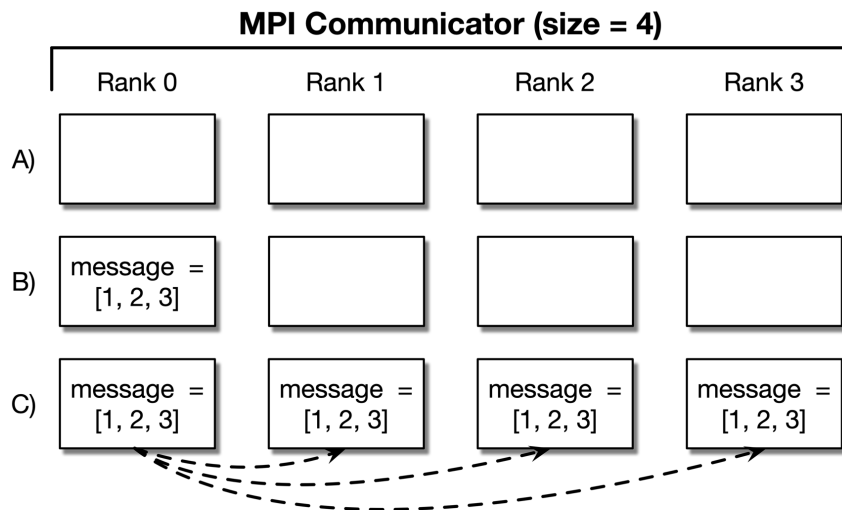
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne 
NATIONAL LABORATORY

REPAST4PY AND MPI

- Repast4Py is implemented in MPI (Message Passing Interface), using the mpi4py Python package.
- MPI applications run on **separate, concurrently** executing processes (**ranks**) that **do not share memory** and the processes communicate by passing **messages**.

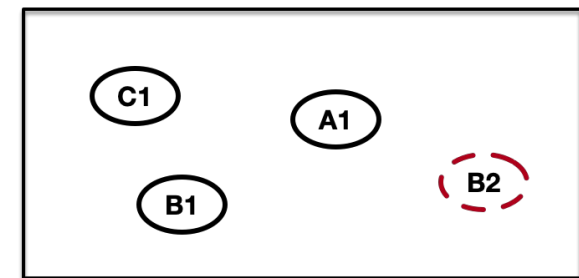
```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 A) size = comm.Get_size()
5 rank = comm.Get_rank()
6
7 print(f"Hello, World! I am rank {rank} of {size}")
8 if rank == 0:
9 B) message = [1, 2, 3]
10 else:
11 message = None
12
13 C) message = comm.bcast(message, root=0)
14 print(f"{rank} received {message}")
15
```



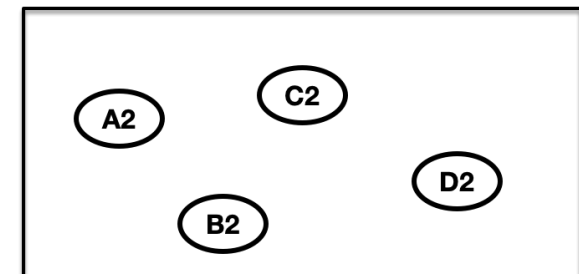
- In a distributed Repast4Py model, agents and their environments run on separate processes that need to be combined into a coherent unified model.

UNIFYING A DISTRIBUTED MODEL: GHOST AGENTS

- A *ghost* agent is a copy of an agent from one process copied onto another.
- Agents *local* to a process can interact with agents from other processes by interacting with the ghosts of those agents.
- User provides code defining the agent data to copy and how to create an agent from that data.
- Repast4Py performs the copy from one process to the other, creating the agent, and adding it to the destination process agent population as a ghost.



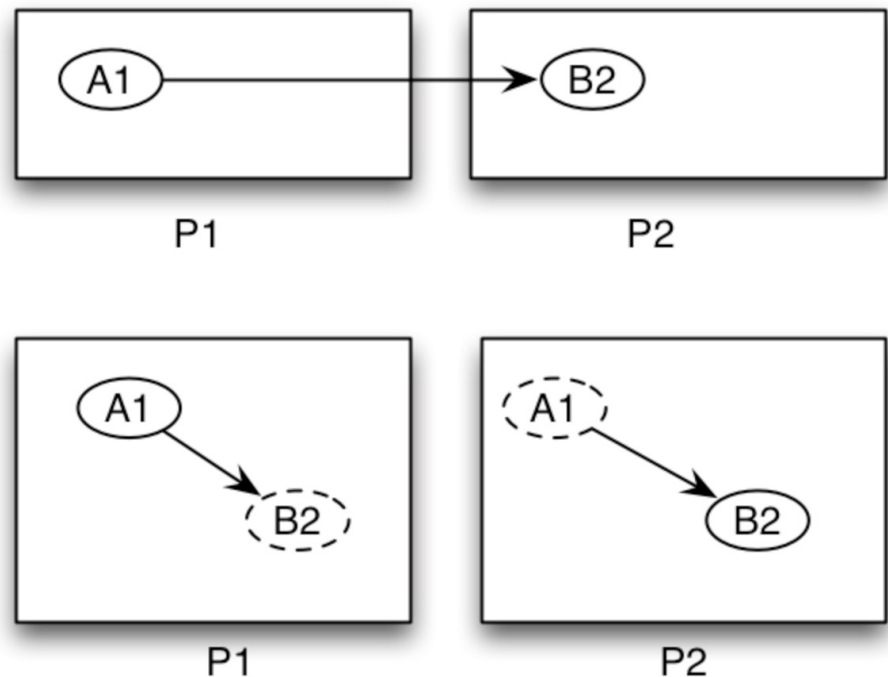
Process P1



Process P2

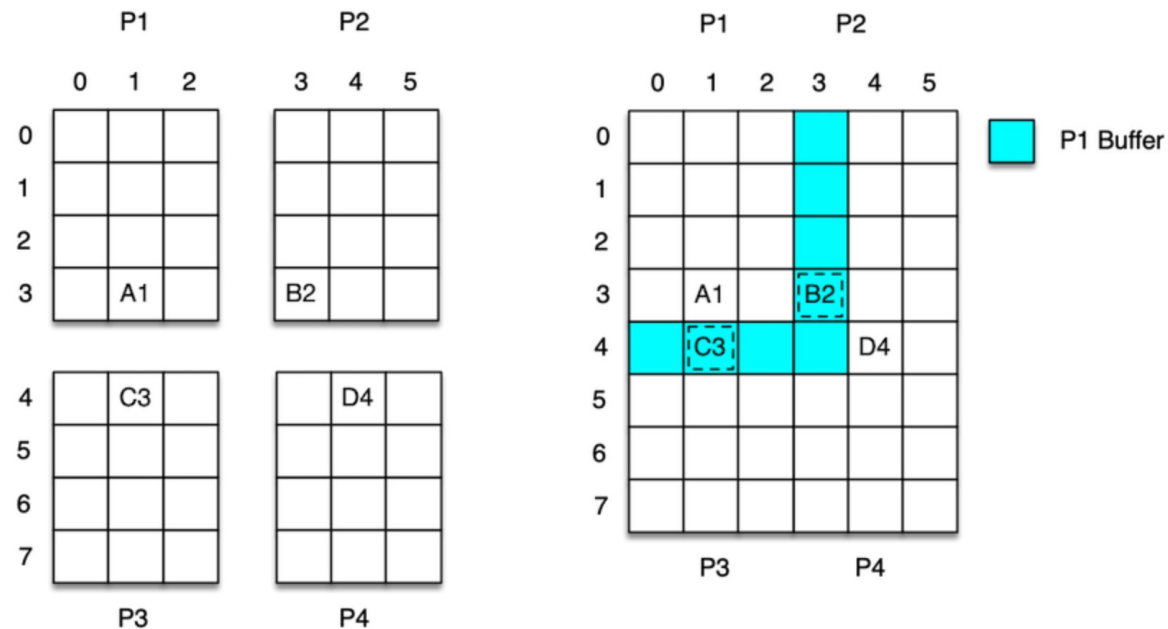
HOW TO DISTRIBUTE ABMS (NETWORKS)

- Ghost agents connect agents across environments, such as networks.
 - Provides access to non-local agents
 - Copying and synchronization of ghost agents and ghost agent states is performed by Repast4Py



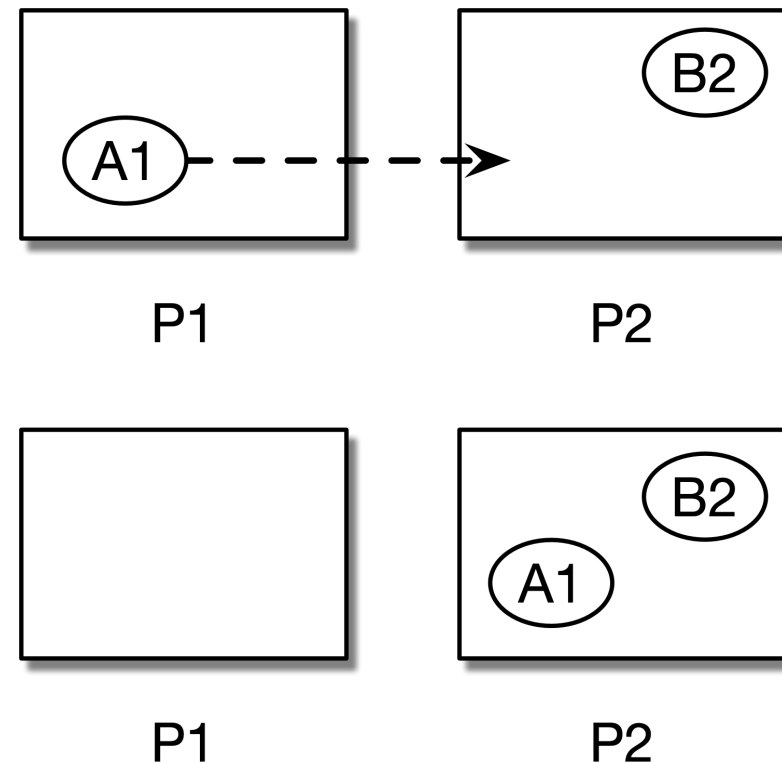
HOW TO DISTRIBUTE ABMS (SPATIAL)

- Buffers are adjacent areas of a global environment duplicated between processes.
- Repast4Py handles the buffer synchronization automatically.



HOW TO DISTRIBUTE ABMS (MOVING)

- Each process is independent and has local agents
 - Agents can move to different processes, e.g., if they pass to part of the space that's owned by another process
 - Copying and synchronization of agents and agent states is performed by Repast4Py
 - Users only needs to provide a minimal amount of code to handle the saving and restoring of agent state



REPAST4PY COMPONENTS: CONTEXTS AND PROJECTIONS

- A **context** is a simple container, based on set semantics, that encapsulates an agent population, and is implemented in pure Python. The equivalence of elements is determined by the agent id.
- A **projection** (an agent environment) takes the population as defined in a context and imposes a structure on it, where the structure is defined by the semantics of the projection. For example, a network projection provides a network structure with agents as nodes.
- Repast4Py implements **shared** contexts and projections as Python classes and modules. **Shared** emphasizes the distributed nature of the simulation, where the global simulation is shared among a pool of processes, each of which is responsible for some portion of it, and stitched together using ghost agents and buffered projections.

SHARED CONTEXT

- A shared context
 - encapsulates and provides access to local agents, ghosts, and projections
 - synchronizes ghost agents and projection buffers
- Each process has a shared context.
- The shared context is implemented by the Repast4Py `repast4py.context.SharedContext` class.
- API documentation provides the details on the various methods for iterating through agents, accessing projections, and synchronizing the model across processes.

Context Module API:

<https://repast.github.io/repast4py.site/apidoc/source/repast4py.context.html>

repast4py.context module

`class repast4py.context.SharedContext(comm)`

Bases: **object**

Encapsulates a population of agents on a single process rank.

A SharedContext may have one or more projections associated with it to impose a relational structure on the agents in the context. It also provides functionality for synchronizing agents across processes, moving agents from one process to another and managing any ghosting strategy.

Parameters: `comm` (*mpi4py.MPI.Intracomm*) – the communicator used to communicate among SharedContexts in the distributed model

add(agent)

Adds the specified agent to this SharedContext.

The agent will also be added to any projections currently in this SharedContext

Parameters: `agent` (*_core.Agent*) – the agent to add

add_projection(projection)

Adds the specified projection to this SharedContext.

Any agents currently in this context will be added to the projection.

Parameters: `projection` (*repast4py.core.SharedProjection*) – the projection add

add_value_layer(value_layer)

Adds the specified value_layer to the this context.

Parameters: `value_layer` (*repast4py.value_layer.SharedValueLayer*) – the value layer to add.

agent(agent_id)

Gets the specified agent from the collection of local agents in this context.

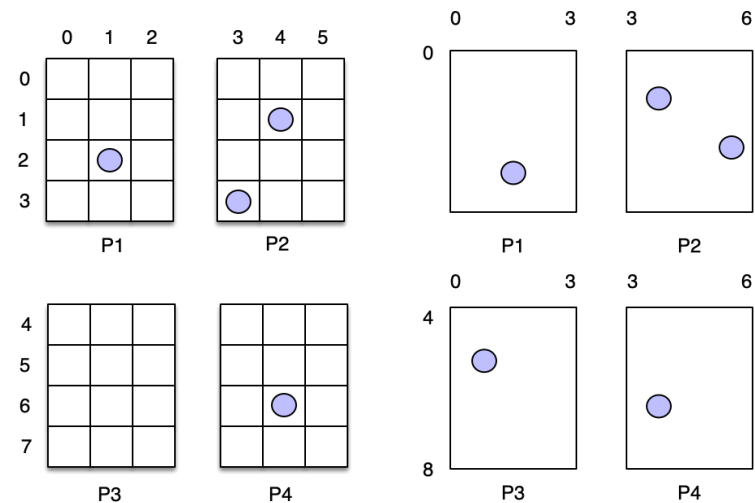
Parameters: `agent_id` – the unique id tuple of the agent to return

Returns: The agent with the specified id or None if no such agent is found.

Return type: *_core.Agent*

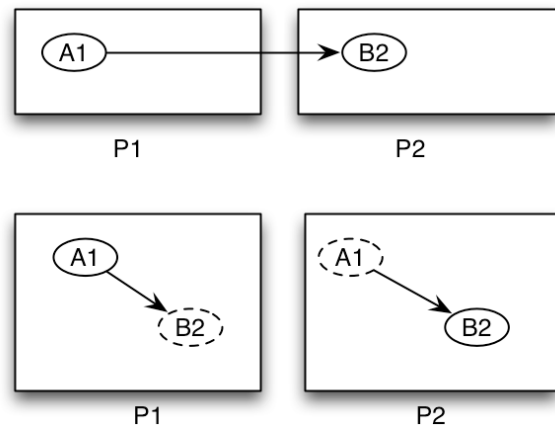
SHARED PROJECTIONS 1

- Shared projections are agent environments partitioned over multiple processes that cooperate with each other and the shared context to create a unified agent environment.
- `space.SharedGrid` is a space in which an agent's location is expressible as a discrete integer coordinate.
- `space.SharedCSpace` is a space in which an agent's location is expressible as a continuous floating point coordinate.



SHARED PROJECTIONS 2

- The SharedNetwork classes implement directed and undirected networks in which agents are the nodes and can form edges with each other. These classes provide typical network functionality such as adding and removing nodes and edges, updating edge attributes and finding network neighbors.
- The SharedValueLayer class is a grid of numeric values that can be retrieved or set via an indexing coordinate.



	0	1	2	3	4	5
0	1.2	0	0	8	1.1	4.2
1	3.2	1	4	7	2.3	6.2
2	3.1	11	3	8.2	8.1	0
3	12	0	33	0	0	0
P1						
	0	1	2	3	4	5
4	13	87	1.2	0	1.2	3.4
5	5	5	6	1.1	0	5
6	8	5	2	3.2	11	0
7	0	0	0	12	0	8
P3						
	0	1	2	3	4	5
0	1.2	0	0	8	1.1	4.2
1	3.2	1	4	7	2.3	6.2
2	3.1	11	3	8.2	8.1	0
3	12	0	33	0	0	0
P2						
	0	1	2	3	4	5
4	13	87	1.2	0	1.2	3.4
5	5	5	6	1.1	0	5
6	8	5	2	3.2	11	0
7	0	0	0	12	0	8
P4						

REPAST4PY AGENTS

- Synchronizing cross-process ghosts and buffers consists of moving and copying agents between processes. This entails:
 - saving the agent state
 - moving or copying that state to another process
 - restoring the agent state as an agent on the destination process.
- For this to work:
 - Agents in a Repast4Py model must extend the `repast4py.core.Agent` class.
 - Agents must have a **unique id tuple with 3 elements**:
 - An integer that uniquely identifies the agent on the rank on which it was created
 - An integer that identifies the model specific type of agent
 - The rank on which the agent was created
 - Agents must implement a **save** method that provides enough of the internal state of the agent that a copy can be created.
 - The modeler must provide a **restore** function that creates an agent from the saved state.

SHARED SCHEDULE

- Events in Repast4Py models, such as when an agent or agents execute their behavior, are driven by a discrete-event schedule.
- Events are scheduled at a particular *tick* that determines when that event will occur.
- In Repast4Py, events are Python functions or classes that implement the `__call__` method (i.e., `Callable`).
- Functions and callables are scheduled for execution on a `SharedScheduleRunner`.
- The `SharedScheduleRunner` synchronizes across processes such that no individual schedule gets ahead of those on other processes.

```
def step():  
    for agent in ctx.agents():  
        agent.do_something()  
  
runner.schedule_repeating_event(1, 1, step)  
runner.schedule_stop(10)
```

LOGGING

- Repast4Py provides multiple logging capabilities.
- ReducingDataSet is appropriate for aggregate-level data within and across ranks.
 - This leverages Python dataclasses. Modelers only modify dataclass instances and Repast4Py takes care of the aggregation.
- TabularLogger is appropriate for individual level data (e.g., logging agent attributes). It can be slow for large amounts of data.
 - Rows are concatenated across processes and written to a file.

```
@dataclass
class Counts:
    total_agents: int
    infected_agents: int
```

HANDS-ON CODING EXERCISE: THE RANDOM WALK MODEL



**U.S. DEPARTMENT OF
ENERGY**

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



THE RANDOM WALK MODEL

- In the Random Walk Model:
 - Agents move at random around a two-dimensional grid
 - Distances are logged
- Each timestep the following occurs:
 1. All the agents (*walkers*) choose a random direction and move one unit in that direction.
 2. All the agents calculate their current distance from their origin location.
 3. The minimum, and maximum distances for that timestep are calculated across all process ranks, and these values are logged.

THE RANDOM WALK MODEL COMPONENTS

- A `Walker` class that implements the agent state and behavior.
- A `Model` class responsible for initialization and managing the model.
- A `restore_walker` function used to create an individual `Walker` when that `Walker` has walked to another process.
- A `run` function that creates and starts the model.
- A main block that allows the model to be run from the command line.
- We will begin with a skeleton and then continue with additional code that implements these components.

BINDER JUPYTER LAB:
[HTTPS://TINYURL.COM/DN37DZUK](https://tinyurl.com/DN37DZUK)



**U.S. DEPARTMENT OF
ENERGY**

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne 
NATIONAL LABORATORY

PERFORMANCE CONSIDERATIONS



**U.S. DEPARTMENT OF
ENERGY**

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne 
NATIONAL LABORATORY

SCALING

- Inter-process communication cost can eventually outweigh the performance advantage of executing fewer agents on each process.
- Models with more agents and more complex agent behavior will typically scale further.
- Investigating where the sweet spot between dividing computational cost among processes and inter-process communication cost lies is an important point of distributed model development.

Process Count	Runtime (sec.)	Speedup	Parallel Efficiency
36	320.15	1x	1
72	170.09	1.88x	0.94
144	91.88	3.48x	0.87
288	71.92	4.39x	0.55

Random Walk Model Strong Scaling

- Additional scaling discussions are available in the conference paper and in an earlier paper:
 - Collier, N. T., J. Ozik, and E. R. Tatara. 2020. “Experiences in Developing a Distributed Agent-Based Modeling Toolkit with Python.” In 2020 IEEE/ACM 9th Workshop on Python for High-Performance and Scientific Computing (PyHPC), 1–12.
<https://doi.org/10.1109/PyHPC51966.2020.00006>

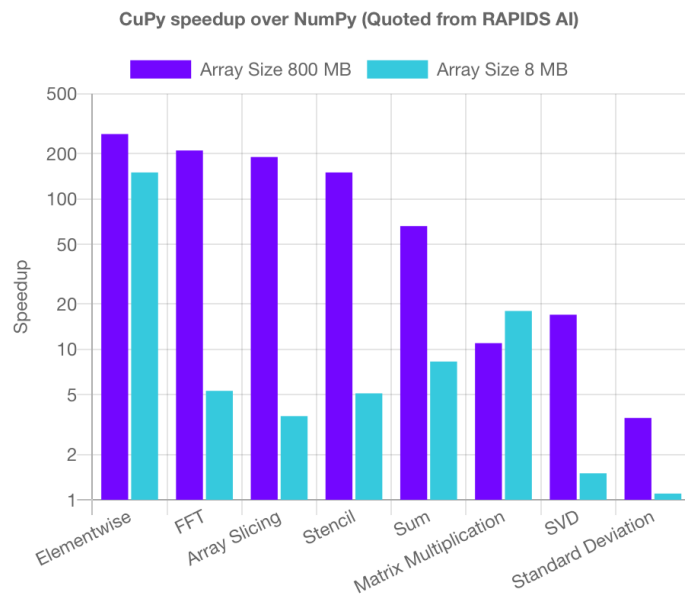
NUMBA

- Numba is a **just-in-time compiler** for Python.
- It can compile certain types of Python functions and classes into **optimized native code** that bypasses the slower Python interpreter.
- It works particularly well for code that is numerically oriented and uses NumPy arrays.
- Additional information on Numba is available at:
<https://numba.pydata.org>

```
spec = [  
    ('mo', int32[:]),  
    ('no', int32[:]),  
    ('xmin', int32),  
    ('ymin', int32),  
    ('ymax', int32),  
    ('xmax', int32)  
]  
  
@jitclass(spec)  
class GridNghFinder:  
  
    def __init__(self, xmin, ymin, xmax, ymax):  
        self.mo = np.array([-1, 0, 1, -1, 0, 1, -1, 0, 1], dtype=np.int32)  
        self.no = np.array([1, 1, 1, 0, 0, 0, -1, -1, -1], dtype=np.int32)  
        self.xmin = xmin  
        self.ymin = ymin  
        self.xmax = xmax  
        self.ymax = ymax  
  
    def find(self, x, y):  
        xs = self.mo + x  
        ys = self.no + y  
  
        xd = (xs >= self.xmin) & (xs <= self.xmax)  
        xs = xs[xd]  
        ys = ys[xd]  
  
        yd = (ys >= self.ymin) & (ys <= self.ymax)  
        xs = xs[yd]  
        ys = ys[yd]  
  
        return np.stack((xs, ys, np.zeros(len(ys), dtype=np.int32)), axis=-1)
```

CUPY

<https://cupy.dev>



HIGH PERFORMANCE WITH GPU

CuPy is an open-source array library for GPU-accelerated computing with Python. CuPy utilizes CUDA Toolkit libraries including cuBLAS, cuRAND, cuSOLVER, cuSPARSE, cuFFT, cuDNN and NCCL to make full use of the GPU architecture. The figure shows CuPy speedup over NumPy. Most operations perform well on a GPU using CuPy out of the box. CuPy speeds up some operations more than 100X. Read the original benchmark article [Single-GPU CuPy Speedups](#) on the RAPIDS AI Medium blog.

CUPY

HIGHLY COMPATIBLE WITH NUMPY & SCIPY

CuPy's interface is highly compatible with NumPy and SciPy; in most cases it can be used as a drop-in replacement. All you need to do is just replace `numpy` and `scipy` with [cupy](#) and [cupyx.scipy](#) in your Python code. The [Basics of CuPy](#) tutorial is useful to learn first steps with CuPy.

CuPy supports various methods, indexing, data types, broadcasting and more.

```
>>> import cupy as cp
>>> x = cp.arange(6).reshape(2, 3).astype('f')
>>> x
array([[ 0.,  1.,  2.],
       [ 3.,  4.,  5.]], dtype=float32)
>>> x.sum(axis=1)
array([ 3., 12.], dtype=float32)
```

CUPY AND ABM

- Agents are rows in a 2D matrix where the columns are the agent attributes.
- Agent attributes can be manipulated in parallel using CuPy / NumPy operations.

```
sus_idx = np.nonzero(self.person_data[:, P_STATE_IDX] == SUSCEPTIBLE)[0]
n_sus = sus_idx.shape[0]
stoe_idx = sus_idx[self.rng.random(n_sus) <= stoe_p]
np.put(self.person_data[:, P_STATE_IDX], stoe_idx, EXPOSED)
```

- When CuPy operations aren't sufficient, custom CUDA kernels can be used.
- This 2D matrix-based design can be combined with Repast4Py components, such as scheduling, logging, and agent representations.

**QUESTIONS?
THANK YOU FOR ATTENDING!**

**JOZIK@ANL.GOV
NCOLLIER@ANL.GOV**